

Design / Architecture Document

ChatGPT Clone Application

Project: ASE 285 ChatGPT Clone

Architecture: React + Node/Express + Socket.IO + MongoDB + OpenAI API + Docker

1. System Overview

The ChatGPT Clone is a full-stack, multi-user AI chat application.

Users can:

- Sign up, log in, and log out
- Create and manage chat sessions
- Send messages and receive AI-generated responses
- Upload files for AI processing
- View persistent, user-specific chat history

The application is containerized with Docker and can be run locally using Docker Compose.

2. Main Goals

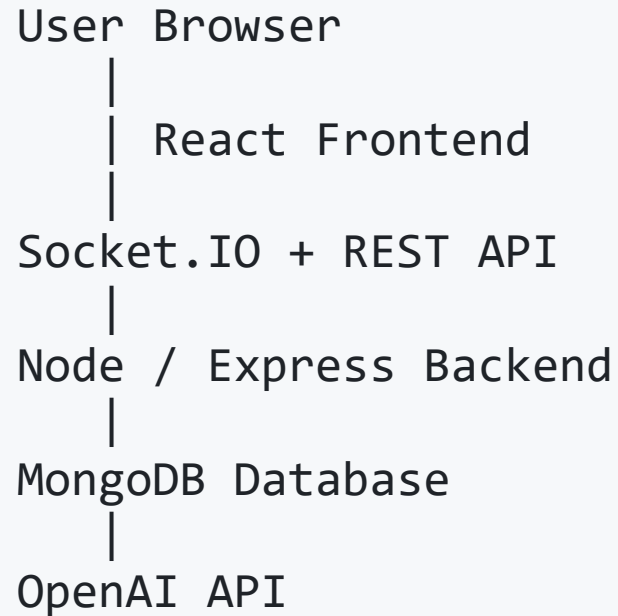
The main goals of the system are to:

- Provide a ChatGPT-style user experience
- Support real-time messaging
- Persist conversations in a database
- Keep each user's chats private
- Allow file attachments to be processed by the AI
- Make the app easier to run through Docker

3. Technology Stack

Layer	Technology
Frontend	React
Backend	Node.js + Express
Real-time Communication	Socket.IO
Database	MongoDB
AI Integration	OpenAI API
Authentication	JWT + bcrypt
File Upload	Multer
Deployment / Runtime	Docker + Docker Compose

4. High-Level Architecture



The frontend handles the user interface.

The backend manages authentication, socket events, file processing, database operations, and AI requests.

5. Frontend Architecture

The frontend is organized into major sections:

```
src/  
  Auth/  
    Login.js  
    Signup.js  
  
  Client/  
    socket.js  
  
  Dashboard/  
    Dashboard.js  
    dashboard.css  
  
  Chat/  
    Chat.js  
    Message.js  
    Messages.js  
    NewMessageInput.js  
  
  Sidebar/  
    Sidebar.js
```

6. Frontend Responsibilities

The React frontend is responsible for:

- Rendering login and signup forms
- Managing the dashboard layout
- Showing chat messages
- Displaying the sidebar and chat list
- Handling chat selection
- Sending messages through Socket.IO
- Uploading files through the backend upload API
- Showing file attachments as clean previews
- Displaying the AI thinking indicator

7. Backend Architecture

The backend is built with Node.js and Express.

Main backend responsibilities:

- Handle signup and login routes
- Hash passwords using bcrypt
- Create and verify JWT tokens
- Authenticate Socket.IO connections
- Store and load user-specific chat sessions
- Handle new chat, rename, delete, and pin events
- Process uploaded files
- Send prompts to the OpenAI API
- Save AI responses to MongoDB

8. Database Design

The application uses MongoDB.

User Collection

```
{  
  username: String,  
  password: String  
}
```

Passwords are stored as hashed values, not plain text.

9. Session Collection

Each chat session belongs to a specific user.

```
{
  userId: ObjectId,
  title: String,
  pinned: Boolean,
  messages: [
    {
      id: Number,
      sender: String,
      content: String,
      attachmentName: String,
      attachmentType: String
    }
  ],
  createdAt: Date,
  updatedAt: Date
}
```

10. Authentication Flow

```
graph TD; A[User enters username/password] --> B[Frontend sends request to /login]; B --> C[Backend validates credentials]; C --> D[Backend returns JWT token]; D --> E[Frontend stores token in sessionStorage]; E --> F[Socket connects using token]; F --> G[Backend identifies the user];
```

User enters username/password
|
Frontend sends request to /login
|
Backend validates credentials
|
Backend returns JWT token
|
Frontend stores token in sessionStorage
|
Socket connects using token
|
Backend identifies the user

JWT authentication is used to protect user-specific chat data.

11. Socket.IO Flow

Socket.IO is used for real-time chat behavior.

Main socket events:

- `get_sessions`
- `new_session`
- `send_message`
- `delete_session`
- `rename_session`
- `toggle_pin`
- `sessions_data`
- `message_received`

12. Message Flow

```
graph TD; A[User types message] --> B[Frontend emits send_message]; B --> C[Backend saves user message]; C --> D[Backend sends content to OpenAI API]; D --> E[OpenAI returns AI response]; E --> F[Backend saves AI response]; F --> G[Frontend receives updated sessions];
```

User types message
|
Frontend emits send_message
|
Backend saves user message
|
Backend sends content to OpenAI API
|
OpenAI returns AI response
|
Backend saves AI response
|
Frontend receives updated sessions

This flow keeps message history persistent and synchronized.

13. File Attachment Flow

```
User selects file
  |
Frontend sends file to /upload
  |
Backend extracts file content
  |
Frontend displays file name as preview
  |
Backend sends extracted content to AI
```

The UI shows a clean attachment preview instead of dumping raw file content into the chat.

14. File Types Supported

The upload feature supports:

- PDF
- TXT
- DOCX
- XLSX / CSV
- Images

Files are processed on the backend before being used by the AI.

15. Sidebar Features

The sidebar supports:

- New chat creation
- Chat selection
- Rename chat
- Delete chat
- Pin/unpin chat
- Search conversations
- Filter conversations
- Sort conversations
- Collapse/expand sidebar
- Resize sidebar width

16. User Session Isolation

The app prevents users from seeing other users' chats by using:

- JWT token authentication
- `socket.userId`
- MongoDB queries filtered by `userId`

Example:

```
Session.find({ userId: socket.userId })
```

This ensures every user only loads their own conversations.

17. Docker Architecture

The app can be run with Docker Compose.

Typical containers:

```
React Frontend  
Node/Express Backend  
MongoDB Database
```

Docker helps make the app easier to build and run consistently across different machines.

18. Docker Run Flow

To run the app locally:

```
docker compose up --build
```

Then open:

```
http://localhost:3000
```

The backend runs on port `5000`, and MongoDB runs inside the Docker network.

19. Testing Strategy

The project includes tests at four levels:

- Unit Tests
- Integration Tests
- Regression Tests
- Acceptance Tests

These tests verify both individual components and major user workflows.

20. Unit Testing

Unit tests cover individual components such as:

- Message display
- File preview display
- Chat input
- Sidebar
- New chat button
- List items
- Login form
- Signup form

These tests confirm that components behave correctly on their own.

21. Integration Testing

Integration tests cover how features work together, including:

- Dashboard loading sessions
- Socket event communication
- Creating chats
- Selecting chats
- Sending messages
- Uploading files
- Renaming, deleting, and pinning chats
- Searching, filtering, and sorting conversations

22. Regression Testing

Regression tests verify that fixed bugs do not return.

Examples include:

- File content should not display in the UI
- File preview should still appear
- Send button should still work
- Attach button should still work
- New chat should still work
- MongoDB `_id` should be used for session selection

23. Acceptance Testing

Acceptance tests verify full user workflows:

- User can sign up
- User can log in
- User can create a chat
- User can send a message
- User can upload a file
- User can log out

These tests check the app from the user's perspective.

24. Security Considerations

Security features include:

- Password hashing with bcrypt
- JWT-based authentication
- User-specific session filtering
- No plain-text password storage
- Socket authentication before loading chat data

These controls protect user data and conversation privacy.

25. Key Design Decisions

Important design decisions:

- Socket.IO was used for real-time communication
- MongoDB was used for flexible chat/session storage
- JWT was used for authentication
- File content is processed internally but shown as a preview
- Docker was used for consistent local setup
- Components were separated by feature area

26. Limitations

Current limitations:

- AI responses depend on internet connection and OpenAI API availability
- File processing is basic and may not handle every complex document
- The app is designed mainly for local/Docker-based use
- Advanced production deployment would require additional environment configuration

27. Future Improvements

Possible future improvements:

- Streaming AI responses
- Better file preview cards
- Drag-and-drop file upload
- User profile settings
- Dark/light mode toggle
- Production deployment with hosted database
- More detailed backend route tests

28. Conclusion

The application provides a full ChatGPT-style experience with:

- Real-time AI chat
- User authentication
- Persistent user-specific conversations
- File attachment support
- Chat management tools
- Docker-based setup
- Multi-level test coverage

